



Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Parky - Marketplace colaborativo de aparcamiento Peer-to-Peer

Peer-to-Peer Collaborative Parking Marketplace

Hugo González Pérez

La Laguna, 15 de abril de 2026

D. **Francisco Javier Rodríguez González**, con N.I.F. 43.618.712-V profesor Asociado de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como tutor

D. **Alejandro Pérez Nava**, con N.I.F. 43.821.179-S profesor Asociado de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como cotutor

C E R T I F I C A (N)

Que la presente memoria titulada:

“Parky - Marketplace colaborativo de aparcamiento Peer-to-Peer”

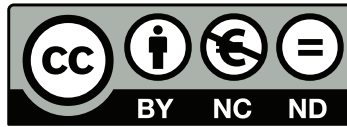
ha sido realizada bajo su dirección por D. **Hugo González Pérez**, con N.I.F 78.824.986-F.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos, firman la presente en La Laguna a 15 de abril de 2026

Agradecimientos

A mi familia, especialmente a mis padres, por su apoyo incondicional y por creer siempre en mí, dándome la fuerza necesaria para superar cada etapa de mi formación académica. Sin su sacrificio y cariño, nada de esto habría sido posible. A mis amigos y compañeros, por los momentos compartidos, las largas jornadas de estudio y por hacer que este camino fuera mucho más ameno. Por último, a la Universidad de La Laguna por brindarme el entorno y las herramientas necesarias para formarme como profesional.

Licencia



© Esta obra está bajo una licencia de Creative Commons
Reconocimiento-NoComercial-SinObraDerivada 4.0
Internacional.

Resumen

En la última década, el incremento del número de vehículos y, sobre todo, del turismo en muchas ciudades canarias ha supuesto una mayor dificultad tanto para el tránsito como para encontrar aparcamiento, sobre todo en las zonas urbanas y en las zonas turísticas. Esto trae problemas tales como la congestión del tráfico, tiempo perdido para los conductores y uso ineficiente del espacio disponible. Al mismo tiempo, numerosas plazas de aparcamiento privadas permanecen vacías durante largos periodos de tiempo.

En este sentido, las plataformas digitales que se sustentan en la economía colaborativa se han transformado en una herramienta útil para conectar personas que tienen un recurso que ofrecer con aquellas que lo necesitan (p. ej., Airbnb, BlaBlaCar, Wallapop).

Por ello, en el presente Trabajo de Fin de Grado se propone el desarrollo de un prototipo de aplicación Full Stack para el alquiler de plazas de aparcamiento privadas. La plataforma operará como un marketplace que conectará a propietarios de plazas (casas particulares o comunidades) con conductores que requieran estacionar su vehículo, facilitando la gestión de reservas y la disponibilidad de los aparcamientos.

Desde el aspecto técnico, el sistema se ha desarrollado con React 18 y Vite como plataforma frontend, TypeScript con modo estricto y Tailwind CSS para el diseño de interfaz. El backend se sustenta en Supabase, un servicio gestionado que proporciona base de datos PostgreSQL, autenticación con OAuth social y almacenamiento de ficheros. La seguridad de los datos se garantiza mediante políticas de Row Level Security (RLS) y triggers PL/pgSQL que aseguran la integridad de las reservas a nivel de base de datos. La visualización cartográfica se implementa con Leaflet. El sistema incluye un motor de precios dinámicos configurable, un módulo de acceso físico digital (SmartAccess) vinculado al estado de cada reserva, y una arquitectura de pagos P2P mediante Stripe Connect que contempla la gestión del IGIC canario. La aplicación se despliega simultáneamente como Single Page Application en Vercel y como aplicación Android nativa mediante Capacitor.

Palabras clave: Marketplace, P2P, aparcamiento, React, Vite, Supabase, Stripe Connect, Capacitor, economía colaborativa, reservas, Row Level Security.

Abstract

In the last decade, the increase in the number of vehicles and, above all, tourism in many Canary Islands cities has led to greater difficulty both for traffic circulation and for finding parking, especially in urban areas and tourist zones. This situation brings problems such as traffic congestion, time lost by drivers, and inefficient use of the available space. At the same time, many private parking spaces remain empty for long periods of time.

In this sense, digital platforms that are based on the sharing economy have become a useful tool to connect people who have a resource to offer with those who need it (e.g., Airbnb, BlaBlaCar, Wallapop).

For this reason, in the present Final Degree Project the development of a Full Stack prototype application for the rental of private parking spaces is proposed. The platform will operate as a marketplace that will connect parking space owners (private homes or residential communities) with drivers who need to park their vehicles, facilitating the management of reservations and the availability of parking spaces.

From a technical perspective, the system was built with React 18 and Vite as the frontend platform, TypeScript in strict mode, and Tailwind CSS for interface design. The backend relies on Supabase, a managed service providing PostgreSQL, social OAuth authentication, and file storage. Data security is enforced through Row Level Security (RLS) policies and PL/pgSQL triggers that guarantee booking integrity at the database level. Map visualisation is handled by Leaflet. The system includes a configurable dynamic pricing engine, a digital physical access module (SmartAccess) linked to booking status, and a P2P payment architecture via Stripe Connect that handles the Canary Islands IGIC tax. The application is deployed simultaneously as a Single Page Application on Vercel and as a native Android application via Capacitor.

Keywords: Marketplace, P2P, parking, React, Vite, Supabase, Stripe Connect, Capacitor, sharing economy, reservations, Row Level Security.

Índice general

Capítulo 1. Introducción	1
1.1. Definición del problema	1
1.2. Justificación	2
1.3. Estado del arte	3
1.3.1. Aplicaciones de zona regulada	3
1.3.2. Plataformas de reserva en aparcamientos comerciales	3
1.3.3. Plataformas P2P de referencia internacional	4
1.3.4. Análisis comparativo	4
1.4. Tendencias del mercado	5
1.4.1. El mercado de aparcamiento en España	5
1.4.2. El mercado en Canarias y el área metropolitana de Tenerife	5
1.4.3. Disponibilidad y precios en el modelo P2P	6
1.5. Objetivos	6
Capítulo 2. Estudio previo	8
2.1. Lenguajes de programación y frameworks	8
2.1.1. Frontend	8
2.1.2. APIs externas de cartografía y geocodificación	11
2.1.3. Backend	11
2.1.4. Base de datos	12
2.2. Entorno de desarrollo	12
2.2.1. Editor de código	12
2.2.2. Control de versiones	13
2.2.3. Diseño de interfaces	13
2.2.4. Herramientas de calidad	13
2.3. Plataformas de despliegue	13
2.3.1. Web: Vercel	13
2.3.2. Android: Capacitor	14
2.4. Análisis de alternativas y decisiones de diseño	14
Capítulo 3. Metodología y análisis de requisitos	17
3.1. Metodología de desarrollo	17
3.1.1. Adaptación iterativa para desarrollo unipersonal	17
3.1.2. Planificación temporal operativa	18
3.2. Actores del sistema	18
3.3. Requisitos funcionales	18
3.3.1. Módulo de Autenticación e Identidad	18
3.3.2. Módulo de Gestión Espacial (Plazas)	20
3.3.3. Módulo de Reservas Transaccionales	20
3.3.4. Módulo de Transacciones Contables (Pagos)	20

3.3.5. Módulos Auxiliares de Calibración	21
3.4. Requisitos no funcionales	21
3.5. Modelo de casos de uso	21
Capítulo 4. Diseño del sistema	26
Capítulo 5. Implementación	27
Capítulo 6. Pruebas y validación	28
Capítulo 7. Conclusiones y líneas futuras	29
Capítulo 8. Summary and Conclusions	30
Capítulo 9. Presupuesto	31
9.1. Sección Uno	31
Apéndice A. Título del Apéndice 1	33
A.1. Algoritmo XXX	33
A.2. Archivo XXY	33
A.3. Algoritmo YYY	34
A.4. Algoritmo ZZZ	34
Apéndice B. Título del Apéndice 2	35
B.1. Otro apéndice: Sección 1	35
B.2. Otro apéndice: Sección 2	35

Índice de figuras

1.1. Impacto de la problemática del aparcamiento (a) y propuesta de optimización mediante Parky (b).	3
2.1. Tecnologías que componen la infraestructura principal <i>Frontend</i> : HTML5, TypeScript, React y Tailwind CSS.	10
2.2. Representación del stack de persistencia y lógica: PostgreSQL y Supabase.	12
3.1. Diagrama de Casos de Uso General intersecando la dependencia cliente/entorno. . .	23

Índice de tablas

1.1. Comparativa de plataformas de aparcamiento	4
1.2. Comparativa de tarifas de aparcamiento regulado por ciudad y zona.	5
2.1. Comparativa de frameworks JavaScript de frontend para el desarrollo de Parky. . . .	10
2.2. Comparativa de plataformas BaaS frente a las necesidades de Parky.	14
2.3. Resumen del stack tecnológico de Parky con versiones de producción.	16
3.1. Cronograma simplificado del desarrollo de infraestructura de Parky.	18
3.2. Identificación de Actores que interactúan directamente en la topología de la plataforma.	19
3.3. Delimitación estricta de las variables y requerimientos no funcionales de la aplicación.	22
3.4. CU-02: Publicar plaza de garaje temporal.	24
3.5. CU-04: Realizar reserva y depósito transaccional.	25
9.1. Presupuesto de Equipos y Licencias	31
9.2. Coste de Mano de Obra	31
9.3. Coste Total del Proyecto	32

Capítulo 1

Introducción

1.1. Definición del problema

No es una novedad que el crecimiento del *parque móvil*¹ y el auge del turismo en las Islas Canarias y sobre todo en Tenerife han provocado que encontrar aparcamiento en prácticamente cualquier lugar sea algo desesperante. En las zonas de alta densidad, como puede ser el (*área metropolitana*)², el problema es aún más acentuado, donde tanto residentes como visitantes diariamente sufren un desequilibrio crítico entre oferta y demanda de espacio urbano.

En muchas vías de Tenerife, este problema se convierte en una desgracia continua y que va a más cada día. Cualquier conductor que transite por la zona que hay entre Santa Cruz de Tenerife y San Cristóbal de La Laguna puede verse atrapado en atascos que, en trayectos de apenas diez kilómetros, agotan la paciencia y degradan significativamente la calidad de vida. Sin embargo, el problema no reside únicamente en la congestión de vías principales como la (*TF-5*)³, sino en la dificultad añadida de encontrar un hueco libre al llegar al destino. De hecho, se estima que hasta un 30 % del tráfico urbano en horas punta está compuesto por vehículos que circulan en bucle buscando estacionamiento, lo que contribuye innecesariamente a la contaminación acústica y a las emisiones de gases de efecto invernadero (*GEI*)⁴.

Es irónico y paradójico el hecho de que, mientras cientos de conductores pierden muchísimo tiempo y gasolina en la búsqueda ineficiente de plazas de aparcamiento, haya numerosas de estas que, ya sea por jornadas laborales o incluso por periodos vacacionales, permanezcan vacías durante gran parte del día o incluso días completos. Este desaprovechamiento de un recurso tan valioso cuando estás frustrado por aparcar pone a la vista una clara ineficiencia en la gestión del espacio disponible.

Es por ello que, a partir de este gran contraste, nace la motivación de Parky y su principal objetivo: Explorar cómo se puede conectar a los propietarios de estas plazas privadas poco usadas con conductores que requieren estacionar su vehículo, facilitando por medio de (*tecnologías digitales*)⁵ una gestión más inteligente y eficiente de los recursos urbanos existentes.

¹**Parque móvil:**El hace referencia al conjunto total de vehículos registrados y en circulación en una determinada región o país.

²**Área metropolitana:**En el contexto de Tenerife, hace referencia principalmente a los municipios de Santa Cruz de Tenerife y San Cristóbal de La Laguna, que concentran una gran parte de la población y actividad económica de la isla.

³**TF-5:**Es la principal autopista del norte de Tenerife, que conecta Santa Cruz de Tenerife con el área metropolitana y los municipios del norte de la isla.

⁴**GEI:**Los gases de efecto invernadero (GEI) son aquellos que contribuyen al calentamiento global al retener el calor en la atmósfera, como el dióxido de carbono (CO₂).

⁵**Tecnologías digitales:**En este contexto se refiere a plataformas digitales y aplicaciones web o móviles que permiten conectar usuarios y gestionar servicios en línea.

1.2. Justificación

La implementación de **Parky** se justifica mediante la convergencia de tres ejes críticos en el contexto urbano actual, integrando factores normativos, socioeconómicos y tecnológicos.

En primer lugar, desde la perspectiva de la **sostenibilidad y presión regulatoria**, el proyecto responde a la obligatoriedad de implementar Zonas de Bajas Emisiones (*ZBE*)⁶ en municipios de más de 50.000 habitantes, como es el caso de Santa Cruz de Tenerife y San Cristóbal de La Laguna. Una parte sustancial del tráfico urbano es generado exclusivamente por la búsqueda de estacionamiento. Facilitar una plataforma (*P2P*)⁷ permite una transición hacia una movilidad más ordenada, reduciendo significativamente las emisiones de gases contaminantes y el ruido en el centro de las ciudades, alineándose así con los objetivos de las denominadas *Smart Cities*⁸.

En segundo lugar, se destaca la **eficiencia social y económica** impulsada por el auge de la economía colaborativa. Este sector no es solo una tendencia social, sino un mercado en plena expansión con una tasa de crecimiento anual compuesta (CAGR) proyectada del 14,22 % hasta el año 2030. En un entorno de inflación creciente, la monetización de activos inactivos cobra una relevancia especial; **Parky** permite a los propietarios obtener ingresos recurrentes de entre 150 € y 300 € mensuales por una plaza infrautilizada, mientras que ofrece al conductor un ahorro drástico frente a las tarifas de rotación comercial (ver Figura 1.1a). El tiempo, como recurso no renovable, se convierte en el principal ahorro para el ciudadano.

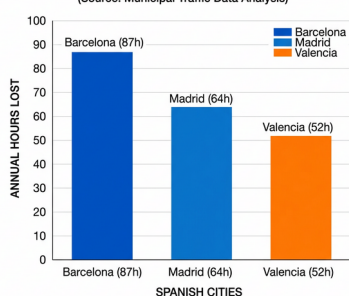
Por último, la **viabilidad tecnológica** de la propuesta es indiscutible. La madurez actual de las infraestructuras de datos en tiempo real y las pasarelas de pago seguras permiten transformar un proceso históricamente manual y opaco en un flujo de uso transparente y eficiente. Esta optimización, comparada con el modelo tradicional de búsqueda aleatoria, se detalla en la Figura 1.1b, demostrando cómo la tecnología actúa como el puente necesario para resolver una ineficiencia logística que ha degradado la calidad de vida urbana durante décadas.

⁶**ZBE:** áreas delimitadas donde se restringe el acceso a los vehículos más contaminantes para mejorar la calidad del aire.

⁷**P2P:** sistema digital que permite a personas o empresas realizar transacciones directas entre sí, sin la necesidad de intermediarios tradicionales como bancos o grandes empresas.

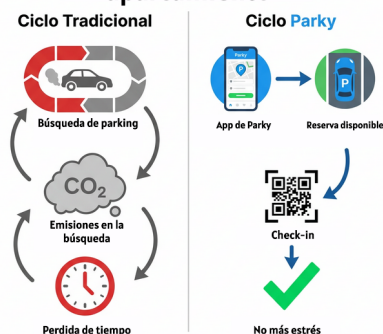
⁸**Smart Cities:** es un entorno urbano que utiliza tecnologías avanzadas, como el Internet de las Cosas (IoT), la inteligencia artificial (IA), el big data y las TIC (Tecnologías de la Información y Comunicación), para mejorar la calidad de vida de sus habitantes, optimizar la gestión de recursos y promover el desarrollo sostenible.

FIGURE 4.2: ANNUAL HOURS LOST SEARCHING FOR PARKING IN KEY SPANISH CITIES (2023)
(Source: Municipal Traffic Data Analysis)



(a) Horas anuales perdidas.

Comparación Búsqueda de aparcamiento



(b) Ciclo Tradicional vs Parky.

Figura 1.1: Impacto de la problemática del aparcamiento (a) y propuesta de optimización mediante Parky (b).

1.3. Estado del arte

Antes de diseñar cualquier sistema, conviene entender bien qué existe ya. En el ámbito del aparcamiento urbano asistido por tecnología hay dos categorías bien diferenciadas que conviene no confundir: las plataformas de gestión de zona regulada y las plataformas de reserva entre particulares. Parky pertenece a la segunda categoría, que en España tiene poca representación y ninguna implantación en Canarias.

1.3.1. Aplicaciones de zona regulada

Las aplicaciones de zona regulada digitalizan el pago de plazas en superficie gestionadas por el municipio. El conductor localiza su zona, introduce la matrícula y paga sin necesidad de un parquímetro físico.

EasyPark opera en más de 4.000 ciudades de 25 países y es el referente del sector. Su modelo depende de convenios con los ayuntamientos; sin ese acuerdo, la plataforma no puede operar. **Telpark**, la alternativa desarrollada por Telefónica en España, sigue la misma lógica B2G⁹ e integra municipios como Madrid, Málaga o Alicante.

Ninguna de las dos permite alquilar o reservar una plaza de titularidad privada. Operan en el espacio público regulado, no en el privado.

1.3.2. Plataformas de reserva en aparcamientos comerciales

Un segundo grupo conecta a conductores con aparcamientos privados ya existentes: parkings de centros comerciales, hoteles o edificios de oficinas que venden capacidad ociosa. **Pardclick** es el mayor

⁹**B2G** (*Business to Government*): modelo de relación comercial entre una empresa privada y una administración pública. En este contexto, las plataformas de zona regulada operan bajo contratos con los ayuntamientos, lo que condiciona su expansión al ritmo de la licitación pública.

operador de este segmento en España, con más de 1.600 instalaciones listadas en 2024. **ElParking** y **Parkimeter** operan en la misma línea con menor cobertura.

El denominador común es que el inventario lo forman instalaciones profesionales con gestión propia, no plazas de particulares. En Canarias, la cobertura de estas tres plataformas es marginal: Parclick lista menos de diez instalaciones en Santa Cruz de Tenerife, todas en el casco comercial.

1.3.3. Plataformas P2P de referencia internacional

El modelo más cercano a Parky es el de **JustPark** [1] en el Reino Unido. Fundada en 2006, permite a cualquier particular publicar su plaza de garaje, entrada de casa o patio privado como oferta alquilable. Actualmente cuenta con más de 50.000 espacios listados y diez millones de usuarios registrados. Su trayectoria demuestra que el modelo P2P de aparcamiento funciona cuando hay masa crítica.

En Estados Unidos, **SpotHero** [2] opera un modelo similar con enfoque en grandes ciudades. Ninguna de las dos plataformas opera en España ni contempla la fiscalidad canaria.

1.3.4. Análisis comparativo

La Tabla 1.1 resume las características principales de cada plataforma frente a Parky.

El análisis deja dos ausencias concretas en el mercado español: no existe ninguna plataforma P2P de aparcamiento entre particulares con presencia en las Islas Canarias, y ninguna de las plataformas revisadas integra un sistema de acceso físico digital vinculado a la reserva. JustPark es el referente conceptualmente más próximo, pero opera exclusivamente en el mercado anglosajón y su modelo no contempla la fiscalidad canaria ni la gestión del IGIC¹⁰.

Parky cubre ese espacio: una plataforma P2P diseñada para el área metropolitana de Tenerife, con acceso digital integrado y adaptada al marco fiscal canario.

Tabla 1.1: Comparativa de plataformas de aparcamiento

Plataforma	P2P	Mapa	Pago integrado	App nativa	Acceso digital	Canarias
EasyPark	No	Sí	Sí	Sí	No	No
Telpark	No	Sí	Sí	Sí	No	No
Parclick	No	Sí	Sí	Sí	No	Parcial
ElParking	No	Sí	Sí	Sí	No	No
JustPark (UK)	Sí	Sí	Sí	Sí	Parcial	No
Parky	Sí	Sí	Sí	Sí	Sí	Sí

¹⁰**IGIC** (Impuesto General Indirecto Canario): tributo indirecto propio de las Islas Canarias que sustituye funcionalmente al IVA peninsular. Su tipo general es del 7 % y el reducido del 3 %. Cualquier actividad económica de prestación de servicios en territorio canario queda sujeta a este impuesto, incluyendo el alquiler de plazas de aparcamiento entre particulares.

1.4. Tendencias del mercado

El problema del aparcamiento no es anecdótico ni local. Hay datos que lo cuantifican con precisión, y esos datos ayudan a calibrar tanto la magnitud de la ineficiencia como el tamaño de la oportunidad que Parky trata de resolver.

1.4.1. El mercado de aparcamiento en España

España tiene uno de los parques vehiculares más densos de Europa. La Dirección General de Tráfico (DGT) [3] registra más de 35 millones de vehículos matriculados en 2022, lo que equivale a aproximadamente 657 vehículos por cada 1.000 habitantes, solo comparable con Italia y Alemania entre los grandes países de la Unión Europea.

Ese volumen tiene un coste real en tiempo. El informe anual de movilidad urbana de INRIX [4] estima que los conductores españoles dedican una media de 24 horas al año exclusivamente a buscar aparcamiento. En las grandes ciudades la cifra sube: Madrid y Barcelona se sitúan entre las diez capitales europeas con mayor tiempo de búsqueda. El coste económico asociado, contabilizando combustible, tiempo y productividad, supera los 1.200 euros por conductor y año en entornos urbanos densos.

Los precios del aparcamiento regulado varían según la ciudad y la zona. En Santa Cruz de Tenerife, la tarifa de zona azul oscila entre 0,60 y 1,20 €/hora. En Madrid (zona A) el precio alcanza los 2,55 €/hora, y en Barcelona sube hasta los 2,75 €/hora en zona de máxima demanda. El aparcamiento en parking comercial se sitúa entre 15 y 35 €/día en los centros urbanos de las capitales peninsulares [3].

Tabla 1.2: Comparativa de tarifas de aparcamiento regulado por ciudad y zona.

Ciudad	Zona	Tarifa (€/hora)	Fuente
Santa Cruz de Tenerife	Zona azul	0,60–1,20	Ayto. Santa Cruz
Madrid	Zona A (SER)	2,55	EMT / Ayto. Madrid
Madrid	Zona B (SER)	1,75	EMT / Ayto. Madrid
Barcelona	Zona 1 (àrea verda)	2,75	BSM / Ajto. Barcelona
Barcelona	Zona 2	1,40	BSM / Ajto. Barcelona

1.4.2. El mercado en Canarias y el área metropolitana de Tenerife

Canarias presenta una situación particular. La insularidad limita las alternativas de transporte público y convierte al vehículo privado en la opción dominante para la mayoría de desplazamientos. El Instituto Canario de Estadística [5] registra en Tenerife más de 420.000 vehículos matriculados en 2023, con una tasa de motorización que supera los 700 vehículos por cada 1.000 habitantes en algunos municipios del área metropolitana, por encima de la media nacional.

La presión sobre el aparcamiento en Santa Cruz de Tenerife y San Cristóbal de La Laguna se explica por la combinación de tres factores: alta densidad residencial en el casco histórico, escasa oferta de aparcamientos públicos subterráneos y concentración de actividad comercial y universitaria. Solo la Universidad de La Laguna, con más de 20.000 estudiantes matriculados, genera una demanda diaria de aparcamiento que el entorno inmediato no puede absorber.

A ello se añade la presión normativa. La Ley 7/2021, de cambio climático y transición energética [6], obliga a los municipios de más de 50.000 habitantes a implantar Zonas de Bajas Emisiones ¹¹ antes del 1 de enero de 2023.

1.4.3. Disponibilidad y precios en el modelo P2P

El precio de alquiler mensual de una plaza de garaje privada en el área metropolitana de Tenerife oscila entre 50 y 120 €/mes según ubicación y características. Frente a los 15-35 €/día de un parking comercial de rotación, incluso la tarifa mensual más alta resulta competitiva para un conductor que necesita aparcar a diario.

Para el propietario, el cálculo también es favorable. Una plaza que genera 80 €/mes suma 960 € anuales por un activo que de otro modo permanece vacío. La plataforma JustPark [1] documenta que sus propietarios en el Reino Unido obtienen entre 1.600 y 3.000 libras anuales por espacio publicado. Extrapolando al mercado canario, el rango estimado se situaría entre 1.000 y 2.500 € anuales, dependiendo de la localización y la frecuencia de uso.

La restricción principal del modelo P2P no es de precio, sino de oferta organizada. No existe en Canarias ninguna plataforma que agregue esas plazas dispersas, las haga visibles en un mapa y gestione la reserva y el pago de forma automática. Esa ausencia es exactamente el espacio que Parky ocupa.

1.5. Objetivos

Objetivo general

Diseñar y desarrollar un prototipo funcional de *marketplace* P2P para el alquiler de plazas de aparcamiento privadas en el área metropolitana de Tenerife, que conecte a propietarios de garajes infrautilizados con conductores que necesitan estacionar, y que resuelva de forma integrada la gestión de reservas, el acceso físico al garaje y el flujo de pagos entre particulares.

¹¹**Ley 7/2021:** establece la obligatoriedad de las ZBE con carácter vinculante. Santa Cruz de Tenerife (aprox. 210.000 hab.) y San Cristóbal de La Laguna (aprox. 160.000 hab.) quedan dentro de su ámbito de aplicación. La restricción de acceso vehicular no elimina la necesidad de aparcar: la desplaza hacia la periferia, donde la oferta de plazas privadas infrautilizadas es mayor.

Objetivos específicos

- OE1.** Implementar un sistema de autenticación empíricamente seguro y de control de acceso basado en roles (RBAC) que permita distinguir la lógica de permisos entre usuarios arrendatarios, propietarios y administradores, con soporte integrado para inicio de sesión abierto (OAuth).
- OE2.** Desarrollar la interfaz de aplicación (*Single Page Application*) enfocada en la experiencia móvil (*Mobile-First*), que incluya búsqueda sobre un mapa interactivo, filtros bidireccionales en tiempo real y rutinas de geocodificación para el registro de nuevas plazas.
- OE3.** Garantizar la privacidad, el entorno *multitenant* y el aislamiento de datos entre los usuarios mediante la definición de políticas de seguridad a nivel de fila (*Row Level Security* - RLS) forzadas directamente en el esquema relacional de la base de datos PostgreSQL.
- OE4.** Implementar un flujo concurrente de reservas que garantice la prevención de solapamientos horarios, asegurando la integridad de las transacciones a través de la programación de *triggers* y procedimientos en base de datos (PL/pgSQL), manteniendo esta lógica agnóstica al lado del cliente.
- OE5.** Modelar la arquitectura frontend y la Experiencia de Usuario (UX/UI) del flujo de checkout, diseñando la integración visual del desglose de cobros tipo *Marketplace* (ej. simulación de Stripe Connect). Esto incluye proyectar cómo el usuario percibe la separación de la tarifa base, la comisión de la plataforma y el cálculo automático del IGIC correspondiente a la jurisdicción canaria.
- OE6.** Integrar rutinas a nivel de sistema para la transición de estados temporales en reservas, además de habilitar las piezas lógicas de un sistema de control de acceso físico integrado con un esquema de valoraciones bidireccionales para incentivar la confianza dentro del sistema *Peer-to-Peer*.
- OE7.** Desplegar la aplicación simultáneamente como un cliente web optimizado y como aplicación móvil, utilizando una arquitectura base compartida en TypeScript y metodologías de unificación de código.

Capítulo 2

Estudio previo

Este capítulo describe las tecnologías que componen Parky y justifica cada elección frente a las alternativas que se consideraron. Para cada herramienta se explica qué función cumple dentro del sistema, por qué se prefirió sobre otras opciones concretas y qué compromisos implica esa decisión.

2.1. Lenguajes de programación y frameworks

La arquitectura separa con claridad dos ámbitos: el *frontend*, ejecutado en el navegador o en el dispositivo Android o iOS, y el *backend*, delegado íntegramente a servicios externos. Esta separación permitió centrar el desarrollo en la lógica de negocio y la experiencia de usuario sin gestionar servidores propios.

2.1.1. Frontend

Para el desarrollo del *frontend*, la plataforma se vertebra a través de diversas tecnologías organizadas en capas funcionales:

- **Base estructural (*HTML5*):** Toda aplicación web, independientemente del framework que use, acaba siendo un documento HTML5¹ que interpreta el navegador. En Parky, el HTML lo genera React en tiempo de ejecución a través de JSX², por lo que el archivo estático `index.html` que sirve Vercel es mínimo; todo el contenido lo inyecta JavaScript tras la carga inicial. Las hojas de estilo que afectan a la estructura visual se generan a partir de las clases de Tailwind CSS y se inyectan también en tiempo de compilación.
- **Código fuente y tipado (*TypeScript 5*):** El código fuente está escrito en TypeScript 5 [7]³. Con `strict: true` en `tsconfig.json`, el compilador detecta incompatibilidades de tipo, propiedades inexistentes y posibles valores nulos antes de que el código llegue al navegador. En un proyecto con formularios complejos, flujos de pago y geolocalización, esta verificación

¹**HyperText Markup Language 5:** Estándar del W3C que define la estructura semántica de los documentos web. Publicado en 2014, incorpora etiquetas semánticas, soporte nativo para audio/vídeo y APIs de geolocalización, entre otras.

²**JavaScript XML:** Extensión de sintaxis que permite escribir marcado similar a HTML directamente en código JavaScript. El compilador (SWC en este caso) lo transforma a llamadas `React.createElement()` antes de ejecutarse.

³**Typescript:** Superconjunto tipado de JavaScript, desarrollado por Microsoft, que añade anotaciones de tipo verificadas en compilación. La versión 5 introdujo decoradores como estándar ECMAScript y mejoras sustanciales en el rendimiento del compilador.

estática elimina categorías enteras de errores que en JavaScript puro solo aparecerían en producción. El compilador configurado es SWC⁴ integrado en Vite.

- **Framework web y reactividad (*React 18*):** La interfaz se construye con React 18.3.1 [8]. El modelo de React es *declarativo* y *reactivo*: el desarrollador describe cómo debe verse la interfaz en función de un estado, y React calcula las diferencias con respecto al estado anterior para actualizar solo los nodos afectados del DOM virtual⁵. La versión 18 introduce el modo concurrente⁶. Se utilizan exclusivamente componentes funcionales con *hooks*⁷
- **Empaquetado y enrutado (*Vite y React Router*):** El empaquetador es Vite 6.4.1 [9]. Durante el desarrollo sirve los módulos directamente como ES Modules nativos, lo que elimina el paso de reempaquetado explícito ante cada cambio y le permite un arranque milisegundo frente al histórico Webpack. Para producción genera un bundle con *tree shaking*⁸ automático. El enrutado usa React Router DOM 7 [10] con *lazy loading*⁹
- **Gestión de estado y formularios (*TanStack y Hook Form*):** La obtención y sincronización de datos del servidor usa TanStack Query v5 [11], que gestiona la caché, los reintentos automáticos y la revalidación en segundo plano. Los formularios (alta de plaza, inicio de sesión) se gestionan con React Hook Form 7, que opera fuera del ciclo de estado de React y evita los costosos re-renders.
- **Estilos y componentes (*Tailwind CSS*):** El sistema de estilos usa Tailwind CSS 4.1.3 [12] con la filosofía *utility-first*¹⁰. Los componentes de interfaz se construyen sobre *Radix UI*, combinada con *shadcn/ui* para facilitar su personalización.

Antes de determinar el stack definitivo, la Tabla 2.1 recoge los factores técnicos que se sopesaron para elegir la arquitectura frontal sobre sus alternativas competidoras directas del mercado:

Más allá de los números de la tabla, el factor decisivo fue **Capacitor**: el adaptador que permite empaquetar la aplicación web como APK nativo de Android. Está sumamente documentado e integrado para React (ya que pertenece a la suite de Ionic). Angular quedó descartado por su poca compatibilidad con Capacitor en comparación con React y por su larga curva de aprendizaje. Vue y Svelte recogen mejoras de rendimiento considerables, pero su ecosistema de utilidades transversales (incluyendo un análogo solvente a TanStack) es menor de cara a integraciones híbridas móviles.

La Figura 2.1 compila de manera visual la tecnología definitiva que conforma el ecosistema *Frontend* actual de la plataforma:

⁴**Speedy Web Compiler:** Transpilador escrito en Rust que reemplaza a Babel en la transformación de TypeScript y JSX. Es entre 20 y 70 veces más rápido que Babel en compilaciones en frío, según los benchmarks publicados por su autor.

⁵**Document Object Model virtual:** Representación en memoria del árbol de componentes que React mantiene para calcular el conjunto mínimo de cambios necesarios en el DOM real del navegador, evitando repintados innecesarios.

⁶**Modo concurrente:** Mecanismo de planificación de renders que permite a React interrumpir un renderizado costoso para atender una interacción del usuario, reanudándolo después. Evita que operaciones como cargar el mapa con cientos de marcadores bloqueen los gestos táctiles.

⁷**Hooks:** Funciones de React que permiten añadir estado, efectos secundarios y lógica reutilizable a componentes funcionales, sin recurrir a clases.

⁸**Tree shaking:** Proceso de eliminación del código que no se referencia en ninguna ruta de importación, reduciendo el tamaño del bundle final.

⁹**Lazy Loading:** Técnica de carga diferida que divide el bundle en fragmentos independientes (*chunks*) y descarga el código de cada pantalla únicamente cuando el usuario la visita por primera vez.

¹⁰**Utility first:** Los estilos se componen en el marcado mediante clases de utilidad atómicas (padding, color, tipografía) en lugar de escribir hojas CSS por componente.

Tabla 2.1: Comparativa de frameworks JavaScript de frontend para el desarrollo de Parky.

Característica	React 18	Vue 3	Angular 17	Svelte 4
Paradigma	Componentes func. + Hooks	Composition API / Options API	MVC + decoradores	Compilado a JS nativo
Modo concurrente	Sí (v18)	No	No	No
App nativa	React Native	No	No	Experimental
TypeScript	Integrado	Integrado	Nativo (obligatorio)	Integrado
Ecosistema	Muy amplio	Amplio	Amplio	Limitado
Mantenedor	Meta	Evan You / comunidad	Google	Vercel / Rich Harris

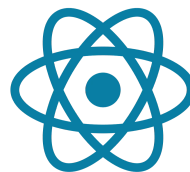


Figura 2.1: Tecnologías que componen la infraestructura principal *Frontend*: HTML5, TypeScript, React y Tailwind CSS.

2.1.2. APIs externas de cartografía y geocodificación

Al tratarse de una aplicación de gestión de aparcamiento, la localización geográfica es una funcionalidad de primer nivel: el usuario busca plazas en un radio de distancia, las visualiza en un mapa interactivo y accede a la dirección exacta. Esto requiere dos capacidades distintas: renderizar mapas y convertir direcciones de texto en coordenadas geográficas.

Leaflet y OpenStreetMap. La visualización cartográfica usa **Leaflet 1.9.4** [13] con el adaptador **React-Leaflet 4.2.1**, sirviendo las teselas de **OpenStreetMap** [14]. Se descartó la API de **Google Maps**¹¹ por su modelo de facturación por volumen. Leaflet es *open source* (licencia BSD-2), sin costes por carga de mapa, y su rendimiento en dispositivos Android de gama media es suficiente para los casos de uso de Parky.

OpenCage Geocoding API. La conversión entre direcciones de texto y coordenadas geográficas (y a la inversa) usa **OpenCage Geocoding API** [15]. Este servicio agrega datos de OpenStreetMap, Geonames y otras fuentes abiertas, con soporte para direcciones canarias. El plan gratuito incluye 2.500 peticiones diarias, suficiente para el volumen esperado durante la fase de prototipado.

2.1.3. Backend

Para la infraestructura de lógica y servicios de apoyo, se evaluó la posibilidad de desarrollar un servidor a medida (*Custom Backend*) mediante Node.js o Python frente al uso de plataformas *Backend-as-a-Service* (BaaS)¹². Finalmente se optó por **Supabase** [16] por los siguientes motivos:

- **Gestión de Autenticación (Auth):** Delega la responsabilidad de la seguridad de las sesiones, el cifrado de contraseñas y los flujos OAuth (Google/Facebook) de forma nativa. Esto elimina la necesidad de implementar y auditar manualmente el sistema de gestión de identidades.
- **Almacenamiento de archivos (Storage):** Proporciona un sistema de gestión de objetos para las imágenes de los garajes y perfiles, con políticas de acceso integradas que permiten que solo el propietario pueda modificar sus recursos.
- **Integración de Pagos P2P (Stripe Connect):** Aunque reside en la lógica de negocio, la arquitectura se apoya en Supabase para orquestar el flujo hacia Stripe Connect. Este modelo permite que la plataforma actúe como intermediaria, reteniendo comisiones y calculando automáticamente el IGIC canario (7 %) sin que los fondos pasen por una cuenta bancaria central de la aplicación, simplificando la fiscalidad.

La elección de una solución BaaS frente a un desarrollo propio se justifica por la necesidad de agilidad y la reducción de la superficie de ataque, permitiendo que un equipo de una sola persona pueda gestionar una infraestructura robusta.

¹¹**Google Maps Platform:** Aplica cargos a partir de 28.000 cargas de mapa al mes (\$200 de crédito gratuito mensual). Para un prototipo en validación con crecimiento impredecible de usuarios, este modelo introduce un riesgo financiero difícil de controlar.

¹²**BaaS (Backend-as-a-Service):** Modelo de computación en la nube donde un proveedor externo aloja y gestiona la infraestructura central del servidor (como almacenamiento, autenticación y bases de datos), permitiendo al desarrollador consumirla a través de APIs documentadas.

2.1.4. Base de datos

El corazón de Parky es su motor de persistencia. Se realizó un estudio comparativo entre el modelo relacional (SQL) y el modelo documental (NoSQL), como se muestra a continuación:

- **Modelo Relacional (*PostgreSQL* [17]):** Elegido finalmente por su capacidad para manejar transacciones complejas. El dominio de las reservas de aparcamiento es inherentemente relacional: una reserva vincula a un usuario con una plaza, que pertenece a un garaje. Las restricciones de integridad (*foreign keys*)¹³ aseguran que no existan reservas huérfanas o duplicadas.
- **Modelo Documental (*MongoDB* [18]):** Descartado debido a que la gestión de la consistencia en reservas concurrentes obligaría a realizar actualizaciones manuales en múltiples documentos, aumentando el riesgo de inconsistencia de datos si una operación falla a mitad del proceso.

Dentro de PostgreSQL (específicamente en su **versión mayor 15**), se han implementado las siguientes características avanzadas:

- **Seguridad a nivel de fila (*RLS*):** Mediante políticas de base de datos, se garantiza que un usuario solo pueda leer sus propias reservas, evitando que errores en el código de la aplicación puedan exponer datos de terceros.
- **Integridad mediante Triggers (*PL/pgSQL*):** Se han programado disparadores que verifican, antes de guardar cualquier reserva, que no exista un solapamiento horario en la misma plaza, garantizando la consistencia incluso si se reciben peticiones simultáneas.



Figura 2.2: Representación del stack de persistencia y lógica: PostgreSQL y Supabase.

2.2. Entorno de desarrollo

2.2.1. Editor de código

El entorno de desarrollo principal es **Visual Studio Code** (VS Code). Para proyectos basados en TypeScript y React, VS Code tiene una ventaja clara sobre alternativas como IntelliJ IDEA: integra el servidor de TypeScript que también usa el compilador, por lo que el análisis de tipos en el editor es idéntico al de la compilación real. El coste en memoria es considerablemente menor que el de IDEs¹⁴

¹³**Foreign Keys (Claves Foráneas):** Columnas ubicadas en una tabla relacional que establecen un vínculo lógico o una dependencia directa hacia la clave primaria de otra tabla, impidiendo, por ejemplo, que se logre crear la reserva de una plaza que realmente no existe.

¹⁴**IDE (Integrated Development Environment):** Entorno de desarrollo integrado que agrupa, en una única aplicación, el redactor de código de un lenguaje junto a un compilador subyacente, automatizador y depurador.

basados en la plataforma IntelliJ, lo que importa cuando se trabaja con el emulador de Android abierto en paralelo.

2.2.2. Control de versiones

El control de versiones usa **Git** [19] con **GitHub** [20] como repositorio remoto. El historial de ramas documenta las fases de desarrollo y facilita la trazabilidad de decisiones técnicas. De cara a un despliegue futuro, GitHub sirve como punto de integración automática con pipelines de Vercel.

2.2.3. Diseño de interfaces

Para la concepción visual pre-programación se empleó **Figma** [21]. Esta valiosa herramienta de diseño UI/UX¹⁵ permitió maquetar pantallas, asimilar paletas de colores y afianzar las distancias entre elementos para su posterior réplica ágil a través de las métricas de *Tailwind CSS*.

2.2.4. Herramientas de calidad

El análisis estático de código recae sobre **ESLint 9** [22] nutrido del plugin `react-hooks`. Además de ser sumamente estricto al arrojar advertencias ante errores tipográficos en variables libres e inconsistencias severas en el formato (código sucio), su verdadero cometido consiste en asegurar la corrección estructural de un archivo localizando dependencias cíclicas y previniendo los re-renderizados infinitos habituales del *DOM*.

2.3. Plataformas de despliegue

2.3.1. Web: Vercel

La versión web está desplegada de forma automatizada en **Vercel** [23], una plataforma *serverless* optimizada para el hospedaje ininterrumpido de aplicaciones SPA. Supone un salto diametral de agilidad frente a alquilar un servidor VPS¹⁶.

De manera imperativa, Vercel automatiza la distribución algorítmica de los empaquetamientos en una CDN Global¹⁷¹⁸ y autogestiona la renovación de las firmas SSL.

¹⁵**UI/UX (User Interface / User Experience)**: Disciplinas enfocadas, respectivamente, en el esqueleto visual de la aplicación y en el flujo narrativo para otorgar confort y un fácil aprendizaje al propio usuario final.

¹⁶**VPS (Virtual Private Server)**: Máquina virtual dedicada alquilada, donde el propio programador es responsable de instalar por su cuenta el sistema operativo o el servidor web subyacente para poder mostrar su plataforma de manera pública.

¹⁷**CDN (Content Delivery Network)**: Red de servidores distribuidos masivamente por el planeta que aloja réplicas y distribuye físicamente el núcleo al nodo geográfico más cercano pertinente al usuario cliente, abaratando la latencia.

¹⁸**Certificado SSL/TLS**: Protocolo universal de autenticación para comprobar la identidad e integridad de un nodo y cifrar simétricamente las llamadas de URL generadas por el mismo, siendo de uso perentorio comercialmente.

Finalmente, para dotar al TFG de identidad empresarial y escapar de los nombres públicos generados sintéticamente, el proyecto de Vercel fue configurado externamente sobre una compra de dominio proporcionada por el registrador **One.com** [24]. Mediante un mapeo estricto a través de registros DNS¹⁹, se instruyó a Vercel a asociar de manera simétrica dicho dominio con la SPA expuesta, garantizando una URL limpia e identitaria lista para los clientes finales.

2.3.2. Android: Capacitor

La aplicación Android se genera con **Capacitor 8.1.0** [25], que encapsula la aplicación web dentro de una *WebView* nativa mediante el patrón *Bridge*²⁰. A diferencia de React Native, que requiere reescribir los componentes de interfaz en primitivas nativas, Capacitor reutiliza las vistas TypeScript/HTML sin modificaciones. El APK final se compila desde Android Studio.

2.4. Análisis de alternativas y decisiones de diseño

Cada decisión tecnológica tiene un coste. Esta sección recoge los casos en los que la elección no fue obvia y los factores que inclinaron la balanza.

Tabla 2.2: Comparativa de plataformas BaaS frente a las necesidades de Parky.

Característica	Supabase [16]	Firebase [26]	AWS Amplify [27]
Motor de base de datos	PostgreSQL (relacional)	Firestore (documental)	DynamoDB (documental)
Consultas relacionales	SQL nativo, JOINS	Limitadas, sin JOINS	Limitadas
Row Level Security	Nativo en PostgreSQL	No disponible	No disponible
Triggers y funciones	PL/pgSQL nativo	Cloud Functions (externo)	Lambda (externo)
Modelo de datos	Normalizado, tipado	Flexible, sin esquema	Flexible, sin esquema
Límite tier gratuito	500 MB / 2 proyectos	1 GB (Spark Plan)	Generoso pero complejo

Firebase y AWS Amplify tienen ecosistemas maduros y escalan bien en bases de datos documentales. El problema es que el dominio de Parky no encaja en ese modelo: una reserva vincula a un usuario con una plaza, en un garaje, durante un intervalo de tiempo, con un pago asociado. Esa cadena de relaciones se expresa en cuatro tablas con *JOINS*; en Firestore exigiría replicar datos en múltiples

¹⁹**DNS (Domain Name System):** Traductores imperceptibles de nomenclatura usados mundialmente que canalizan un nombre de empresa legible para humanos como una dirección IP identificable para el clúster interno de conmutadores interconectados en Internet.

²⁰Patrón de comunicación que traduce llamadas desde el código JavaScript de la *WebView* a la API nativa de Android (Java/Kotlin), permitiendo acceder a funcionalidades del dispositivo como la cámara, el almacenamiento local o la barra de estado.

documentos y aceptar que una actualización parcial deje el sistema en estado inconsistente si el dispositivo pierde conexión a mitad del proceso.

El coste real de usar Supabase es el límite del plan gratuito: 500 MB de base de datos y pausas automáticas tras 7 días de inactividad. Para un prototipo universitario eso es suficiente, pero una versión comercial necesitaría el plan Pro (\$25/mes). Ese coste está recogido en el plan de viabilidad del capítulo 7.

Tabla 2.3: Resumen del stack tecnológico de Parky con versiones de producción.

Capa	Tecnología	Función	Versión
Lenguaje base	TypeScript	Tipado estático sobre JavaScript	5.9.3
Estructura web	HTML5 + JSX	Marcado semántico generado por React	—
Framework UI	React	Componentes reactivos declarativos	18.3.1
Empaquetador	Vite + SWC	Compilación y bundling optimizados	6.4.1
Enrutado	React Router DOM	Navegación SPA con lazy loading	7.12.0
Estado servidor	TanStack Query	Caché y sincronización de datos	5.90.20
Formularios	React Hook Form	Formularios de alto rendimiento	7.71.0
Estilos	Tailwind CSS	Utilidades CSS utility-first	4.1.18
Componentes UI	Radix UI + shadcn/ui	Primitivas accesibles sin estilos opacos	—
Mapas	Leaflet + React-Leaflet	Visualización cartográfica OpenStreetMap	1.9.4 / 4.2.1
Geocodificación	OpenCage Geocoding API	Conversión dirección → coordenadas	—
BaaS	Supabase	Auth, base de datos, almacenamiento	2.90.1
Base de datos	PostgreSQL	Motor relacional con RLS y PL/pgSQL	15
Pagos	Stripe Connect	Marketplace P2P con separación IGIC	—
App nativa	Capacitor	Empaquetado APK para Android	8.1.0
Despliegue web	Vercel	CDN con CI/CD automático desde GitHub	—

Capítulo 3

Metodología y análisis de requisitos

3.1. Metodología de desarrollo

La construcción de un sistema de software moderno requiere una metodología que estandarice el ciclo de vida del desarrollo. Para la concepción de *Parky*, se descartaron aproximaciones tradicionales en cascada (*Waterfall*) debido a la incertidumbre inherente al dominio funcional durante las primeras fases de diseño.

En lugar de definir unos requisitos inmutables por adelantado, el proyecto adopta **Scrum** [28], un marco de trabajo fundamentado en los pilares de la inspección, la transparencia y la adaptación de la *metodología ágil*. El uso del enfoque ágil facilita enormemente las correcciones de rumbo, la incorporación agresiva de nuevas librerías al ecosistema (como *Stripe Connect*) o la supresión técnica de rutas innecesarias sin destruir el esqueleto del proyecto.

3.1.1. Adaptación iterativa para desarrollo unipersonal

Scrum presupone la existencia de un equipo compuesto por diferentes roles (*Product Owner*, *Scrum Master* y desarrolladores), lo cual resulta inviable en un Trabajo de Fin de Grado desarrollado de manera completamente individual. Por consiguiente, se ha ejecutado una adaptación minimalista, donde convergen todos los roles en una única persona y se prescinde de ceremoniales síncronos diarios como los *Daily Standups*.

El trabajo se ha organizado de manera estricta siguiendo un paradigma iterativo:

1. **Pila del producto (*Product Backlog*):** Los requisitos y funcionalidades, organizados como un inventario ordenado por niveles de prioridad de alto nivel en un tablero digital *Kanban* mediante el uso de *GitHub Projects*.
2. **Sprints modulares:** Iteraciones limitadas a dos semanas, enfocadas exclusivamente a un único módulo de alto nivel (por ejemplo: Sprints dedicados exhaustivamente al sistema de Auth y a la creación de bases de datos antes de enlazar la interfaz móvil).
3. **Retrospectiva continua:** Revisiones pragmáticas previas a cambiar de *Sprint* orientadas a limpiar y refactorizar componentes y revisar vulnerabilidades incrustadas en las APIs, evaluando qué impedimentos relantan artificialmente el tránsito funcional de los módulos del proyecto.

3.1.2. Planificación temporal operativa

El cronograma del proyecto quedó segmentado temáticamente para maximizar la cobertura global, comenzando con una fuerte base sobre la gestión de sesiones abstractas y extendiéndose al entorno del GPS nativo.

Tabla 3.1: Cronograma simplificado del desarrollo de infraestructura de Parky.

Módulos de Desarrollo	Mes 1	Mes 2	Mes 3	Mes 4
Planitud e Identidad UI (Figma/Taiw lind)	■			
Identidades: Auth y Base de Datos (Supabase)	■	■		
Lógica núcleo: Mapa y Plazas		■	■	
Lógica núcleo: Sistema de Reservas y Notificaciones			■	■
P2P: Integración Fiscal y Stripe				■
Despliegue Multiplataforma (Vercel y Capacitor)				■

3.2. Actores del sistema

La delimitación de los perfiles que interactúan con una misma plataforma acota significativamente las fronteras y los permisos funcionales permitidos. A diferencia de las infraestructuras genéricas horizontales (como un foro o una red social tradicional de mensajería), en *Parky* existe una dicotomía estricta de responsabilidades entre la oferta privada (arrendadores poseedores del bien material) y la demanda esporádica (arrendatarios usuarios de infraestructuras rodadas). Cada perfil accede únicamente a su propio grupo de información autorizado vía *Row Level Security* sobre la entidad de 'Profiles' abstracta.

3.3. Requisitos funcionales

Los requisitos funcionales detallan las operaciones explícitas que el sistema informático debe ser capaz de procesar sin alterar su comportamiento estándar. Para estructurarlo armónicamente, se han categorizado ateniéndonos a los ejes troncales de los perfiles y espacios de negocio previstos arquitectónicamente.

3.3.1. Módulo de Autenticación e Identidad

- **RF-01 (Registro e inicio multiplataforma):** El sistema debe permitir el alza y login unificado a través de validación semántica delegada (OAuth2 por Google) y un canal nativo (Email y contraseña).

Tabla 3.2: Identificación de Actores que interactúan directamente en la topología de la plataforma.

Actor identificado	Descripción, propósito y alcance en el sistema
Arrendador	Propietario legal que publica los excedentes de garajes, decidiendo de manera vinculante precio e inventarios. Cuenta con el privilegio para alta de nuevas plazas con adjuntos visuales, edición, administración contable de su estado e intercepción/aceptación directa sobre todas las reservas entrantes en el tiempo para sus activos. Cobra indirectamente a través del balance derivado en <i>Stripe Connect</i> .
Arrendatario	Conductor o usuario estático que precisa alquilar un segmento temporal en un garaje específico. Su ámbito computacional es la carga visual del mapa vectorial para el filtrado espacial y visualización de recursos libres, petición directa, emisión de pagos de tarjeta blindados, y emisión posterior de una valoración obligatoria sujeta en base al activo reservado.
Administrador	Represente directo del sistema corporativo de mantenimiento. Actor silencioso fuera del código nativo cliente, y el cual orquesta permisos globales visuales, suspensión directa de identidades y administración global de cuotas y depuración de la Base de Datos mediante el uso privado abstracto del portal RLS nativo incrustado en el <i>dashboard</i> de Supabase.
Sistema	Componente inanimado automático. Conformado tecnológicamente como el consorcio orquestado que vincula la computación invisible: procesa transacciones (<i>Stripe</i>), geocodifica (<i>OpenCage</i>) y autentifica identidades (<i>OAuth</i>) emitiendo notificaciones estáticas mediante base de datos.

- **RF-02 (Acaparamiento de rol):** Inmediatamente posterior al primer login temporal en la plataforma, el sistema debe derivar la vista hasta forzar la selección binaria e intransitiva en base de datos del perfil comercial principal (*Arrendador* o *Arrendatario*).

3.3.2. Módulo de Gestión Espacial (Plazas)

- **RF-03 (Publicación y edición contable):** El arrendador autenticado ostentará potestad única para dar de alta sus propios activos, debiendo acompañarlos de variables obligatorias como geoposición precisa, descripción textual, importe fraccionado mínimo y adjuntos fotográficos. Del mismo modo podrá alterarlos o darlos de baja lógicamente.
- **RF-04 (Filtrado cartográfico dinámico):** El sistema ofrecerá una capa visual interactiva usando renderizado de *Leaflet* incrustado, devolviendo y filtrando *markers* (plazas) cuyo estado sea libre y converja asimétricamente en el polígono visualizado por la cámara del dispositivo móvil.

3.3.3. Módulo de Reservas Transaccionales

- **RF-05 (Emisión de solapa temporal):** El usuario arrendatario dispondrá de una vista de petición con campos de inicio y finalización por fechas solapables.
- **RF-06 (Ciclo de vida intermedio):** Transversalmente, el sistema dictaminará una serie de barreras para la integridad del objeto de reserva: pendiente → confirmada → en curso → completada → cancelada.
- **RF-07 (Confirmación arbitratia):** Toda reserva generada caerá en la bandeja del arrendador en estado pendiente, requiriendo obligatoriamente un control por acción manual por su parte para habilitar tecnológicamente el desembolso e inicio de cobro.

3.3.4. Módulo de Transacciones Contables (Pagos)

- **RF-08 (Pasarela blindada):** Tras la aceptación semántica, la plataforma actuará la reserva interina en estado confirmatorio y enrutará los fondos a la pasarela opaca gestionada por *Stripe*. La comisión matemática impuesta por la plataforma se retiene automáticamente antes del desembolso al proveedor.
- **RF-09 (Resiliencia de reembolsos):** El sistema revertirá íntegramente las transacciones si el evento es anulado formalmente antes del periodo ventana o ante caídas generalizadas de red forzadas, actuando directamente contra el API bancaria.

3.3.5. Módulos Auxiliares de Calibración

- **RF-10 (Valoración póstuma):** Para evitar engaños y fomentar la confianza, los usuarios podrán valorar paramétricamente mediante un ranking ordinal (1 a 5 estrellas) un activo únicamente si validan que la transacción ha sido formalmente completada por ambas partes.
- **RF-11 (Alarma activa proveedor):** El sistema generará activamente una notificación inter-proceso para alertar a un arrendador desconectado cada vez que converja una nueva petición síncrona en sus activos listados.
- **RF-12 (Alarma reactiva cliente):** Idénticamente para la otra perspectiva, los arrendatarios serán alertados visualmente del éxito o rotundo declive provocado tras confirmar/rechazar su bloque de tiempo reservable.

3.4. Requisitos no funcionales

Los requisitos no funcionales moldean estructuralmente cómo la base de operaciones aborda la calidad interna, condicionando restricciones de velocidad, escalabilidad e inyección continua de código. Para Parky se abordan en base a la estandarización paralela para ingenierías de software formales.

3.5. Modelo de casos de uso

El sistema global se disecciona de manera analítica para representar gráficamente las interacciones abstractas de los usuarios sobre la entidad invisible (el sistema). En la topología ilustrada (Figura 3.1), queda latente la partición de las capacidades atadas al grupo de pertenencia de la identidad analizada en el entorno.

Del esquema de dependencias genéricas se pueden entrelazar las operaciones de negocio más voluminosas e interceptables por el producto en fase de producción e ingeniería de software.

Tabla 3.3: Delimitación estricta de las variables y requerimientos no funcionales de la aplicación.

Categoría Crítica	Requisito explorable y restricciones asociadas
Rendimiento	Puesto que la interacción es a través de redes a menudo inestables, el tiempo de renderizado de la topología web matriz (antes de hidratar y ejecutar lógica local) no sobrepasará un tiempo teórico de 3 segundos bajo una conectividad 4G de latencia media.
Seguridad	La barrera arquitectónica para consultar una base de datos en clientes web/móviles requiere ineludiblemente la creación de políticas SQL de filtrado (<i>Row Level Security</i>) en Supabase e interceptación asíncrona por JSON Web Tokens en cada capa.
Usabilidad	A nivel de percepción y contrastes, la interfaz heredaré la librería <i>Tailwind CSS</i> garantizando estricta responsividad escalar en anchos variables pasando las validaciones de contraste W3C bajo la categoría estándar WCAG 2.1 de nivel AA para la inclusión de usuarios monocromáticos.
Portabilidad técnica	Dado que se programa una arquitectura desacoplada React (SPA), este paquete tiene obligatoriamente que operar con la envoltura asíncrona de Capacitor para conformar aplicaciones compiladas nativas aptas y publicables para clientes tanto del entorno <i>Apple iOS</i> como de <i>Android</i> .
Disponibilidad	Los proveedores infraestructurales seleccionados en la comparativa (Vercel Node para los <i>assets</i> y Supabase PostgreSQL para la computación) exigen formalmente un Acuerdo de Nivel de Servicio mínimo del 99,5 % de <i>uptime</i> .
Escalabilidad horizontal	Al desvincularse Parky temporalmente de infraestructuras cerradas tipo VPS por una nube tipo Serverless, la demanda abrupta de clientes se apacigua replicando instancias computacionales por <i>Edge Functions</i> subyacentes, aislando del estrés al código primario programado que se publicará.
Mantenibilidad	El desarrollo sigue estándares rigurosos. Todo parámetro, variable y función queda incursionado con <i>TypeScript Strict</i> asegurando retro-compatibilidad, minimizando fuertemente la incursión de <i>Undefined Types</i> en el entorno de ejecución final del programa.



Figura 3.1: Diagrama de Casos de Uso General intersecando la dependencia cliente/entorno.

Tabla 3.4: CU-02: Publicar plaza de garaje temporal.

ID y Nombre	CU-02: Publicar plaza de garaje
Actor principal	Arrendador
Precondiciones	El actor debe encontrarse correctamente identificado en el sistema (JWT activo) y su perfil primario fijado formalmente como Arrendador.
Postcondiciones	La nueva plaza quedará inscrita de forma persistente como registro público asimilable lógicamente, emitiéndose un refresco automático en todo cliente móvil en la geozona de la plaza.
Flujo principal	1. El arrendador accede formalmente al tablero lateral de sus activos. 2. Completa un formulario de alta iterativo cediendo datos técnicos y adjuntos. 3. Verifica la <i>chincheta</i> paraxial geoposicional para enmarcar su latitud/longitud real en la edificación. 4. Confirma su inserción ejecutando una mutación GraphQL a la API. 5. El sistema notifica asimétricamente la subida exitosa.
Flujo alternativo	Si los campos referidos al precio están corrompidos matemáticamente o la subida de los <i>Buckets</i> fotográficos aborta prematuramente, el núcleo asume pánico bloqueando el registro a Base de Datos y reflejando una alarma modal roja en el origen <i>frontend</i> .

Tabla 3.5: CU-04: Realizar reserva y depósito transaccional.

ID y Nombre	CU-04: Realizar reserva y pago
Actor principal	Arrendatario
Precondiciones	El actor identificable debe poseer acceso asíncrono y los metadatos visuales de la plaza interceptada perimetralmente asimilados.
Postcondiciones	La entidad contable pasa al limbo bajo estado pendiente a expensas puramente de la réplica asimétrica del arrendador, imposibilitándose a otros sobrecribir la brecha cronometrada.
Flujo principal	1. El arrendatario clicla el activo libre desde su interfaz virtual. 2. Diligencia obligatoria del selector de periodos por minutos/horas exactas. 3. El sistema valida criptográficamente solapamientos (triggers SQL en caliente). 4. Se levanta la terminal asíncrona de Stripe forzando al cobro preventivo. 5. Retorno en bucle y validación definitiva confirmando a ambos perfiles involucrados en los buzones nativos.
Flujo alternativo	Si la tarjeta sufre retención severa bancaria o la plaza fue capturada paralelamente por un nodo esclavo ajeno durante el <i>lag</i> estético, remite la inyección contable e informa con una barrera temporal.

Capítulo 4

Diseño del sistema

Los capítulos intermedios sirven para cubrir los siguientes aspectos: antecedentes, problemática o estado del arte, objetivos, fases y desarrollo del proyecto.

En el Capítulo 1 se comentó lo que [29] dijo al respecto. Aquí vamos a profundizar en una de sus afirmaciones más controvertidas:

«Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.» (Albert Einstein)

Es decir que «erat ac sagittis sempe», lo que se ilustra en el esquema de la ??.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Capítulo 5

Implementación

Capítulo 6

Pruebas y validación

Capítulo 7

Conclusiones y líneas futuras

Este capítulo es obligatorio. Toda memoria de trabajo de fin de grado debe incluir unas conclusiones y unas líneas de trabajo futuro.

Capítulo 8

Summary and Conclusions

This chapter is compulsory. The memory should include an extended summary and conclusions in English.

Capítulo 9

Presupuesto

Este capítulo es obligatorio. Toda memoria de trabajo de fin de grado debe incluir un presupuesto.

9.1. Sección Uno

Tabla 9.1: Presupuesto de Equipos y Licencias

Descripción	Cantidad	Coste (€)
Portátil	1	900,00
Licencia de Software de Desarrollo (IDE)	1	100,00
Licencia de Software de Diseño Gráfico	1	50,00
Compra de Componentes Adicionales	1	150,00
Servicios en la Nube	12 meses	240
Subtotal de Equipos y Licencias		1440,00

Tabla 9.2: Coste de Mano de Obra

Descripción	Horas	Coste (€)
Precio por Hora		20,00
Total de Horas de Trabajo	100	
Costo Total del Trabajo Humano		2000,00

Tabla 9.3: Coste Total del Proyecto

Descripción	Coste Total (€)
Subtotal de Equipos y Licencias	1440,00
Costo Total del Trabajo	2000,00
Coste Total del Proyecto	3440,00

Apéndice A

Título del Apéndice 1

A.1. Algoritmo XXX

Ejemplo de código con coloreado de sintaxis:

```
1 #include <iostream>
2
3 int main()
4 {
5     // Imprime "Hello, world!" en la consola
6     std::cout << "Hello, world!\n";
7     return 0;
8 }
```

A.2. Archivo XXY

Ejemplo de JSON usando el mismo entorno de coloreado de sintaxis:

```
1 {
2     "nombre": "John Doe",
3     "edad": 30,
4     "ciudad": "Nueva York",
5     "hobbies": [
6         "lectura",
7         "jardinería",
8         "ciclismo"
9     ],
10    "empleo": {
11        "título": "Ingeniero de Software",
12        "empresa": "TechCorp"
13    }
14 }
```

A.3. Algoritmo YYY

Este es el clásico entorno verbatim, sin coloreado pero con fuente monoespaciada:

```
/*****
 *
 * Fichero .h
 *
 *****/
 *
 * AUTORES
 *
 * FECHA
 *
 * DESCRIPCION
 *
 *****/
```

A.4. Algoritmo ZZZ

Ejemplo de entorno para describir algoritmos en pseudocódigo:

Algoritmo A.1: Cálculo del factorial de un número

```
1: Entrada: Un entero  $n$ 
2: Salida: El factorial de  $n$ 
3: function FACTORIAL( $n$ ) ▷ El factorial de  $n$ 
4:   if  $n \leq 1$  then
5:     return 1 ▷ El factorial de 0 o 1 es 1
6:   else
7:     return  $n \times$  FACTORIAL( $n-1$ )
```

Otra forma de describir algoritmos es utilizar entornos `lstlisting` y emplear una sintaxis de pseudocódigo similar a alguno de los lenguajes soportados por este paquete, como Python o Pascal.

Apéndice B

Título del Apéndice 2

B.1. Otro apéndice: Sección 1

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

B.2. Otro apéndice: Sección 2

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Bibliografía

Las siguientes referencias bibliográficas se presentan en orden de aparición en el texto.

- [1] JustPark Parking Ltd. *About JustPark*. JustPark Parking Ltd. 2023. URL: <https://www.justpark.com/about/> (visitado 20-11-2024).
- [2] SpotHero Inc. *SpotHero: Park Smarter*. SpotHero Inc. 2023. URL: <https://spothero.com/about> (visitado 20-11-2024).
- [3] 20minutos.es. *La DGT confirma que nunca ha habido más vehículos matriculados en España*. 20minutos.es. 2026. URL: https://www.20minutos.es/motor/actualidad/dgt-confirma-nunca-ha-habido-mas-vehiculos-matriculados-espana_6954697_0.html (visitado 14-04-2026).
- [4] INRIX Inc. *INRIX 2022 Global Traffic Scorecard*. INRIX Inc. 2022. URL: <https://inrix.com/scorecard/> (visitado 10-10-2024).
- [5] Instituto Canario de Estadística. *Estadística del Parque de Vehículos de Canarias 2023*. Gobierno de Canarias. 2023. URL: <https://www.gobiernodecanarias.org/istac/> (visitado 10-10-2024).
- [6] Jefatura del Estado. *Ley 7/2021, de 20 de mayo, de cambio climático y transición energética*. 2021. URL: <https://www.boe.es/eli/es/l/2021/05/20/7> (visitado 10-10-2024).
- [7] Microsoft Corporation. *TypeScript: JavaScript with syntax for types*. Microsoft Corporation. 2024. URL: <https://www.typescriptlang.org> (visitado 10-10-2024).
- [8] Meta Platforms, Inc. *React – The library for web and native user interfaces*. Meta Platforms, Inc. 2024. URL: <https://react.dev> (visitado 10-10-2024).
- [9] Evan You y Vite Contributors. *Vite – Next Generation Frontend Tooling*. Vite Contributors. 2024. URL: <https://vitejs.dev> (visitado 10-10-2024).
- [10] Remix Software, Inc. *React Router – Declarative routing for React*. Remix Software, Inc. 2024. URL: <https://reactrouter.com> (visitado 10-10-2024).
- [11] TanStack. *TanStack Query – Powerful asynchronous state management for TypeScript/JavaScript*. TanStack. 2024. URL: <https://tanstack.com/query> (visitado 10-10-2024).
- [12] Adam Wathan y Tailwind Labs, Inc. *Tailwind CSS – A utility-first CSS framework*. Tailwind Labs, Inc. 2024. URL: <https://tailwindcss.com> (visitado 10-10-2024).
- [13] Vladimir Agafonkin. *Leaflet – an open-source JavaScript library for mobile-friendly interactive maps*. 2024. URL: <https://leafletjs.com> (visitado 10-10-2024).
- [14] OpenStreetMap Foundation. *OpenStreetMap*. OpenStreetMap Foundation. 2024. URL: <https://www.openstreetmap.org> (visitado 10-10-2024).
- [15] OpenCage GmbH. *OpenCage Geocoding API*. OpenCage GmbH. 2024. URL: <https://opencagedata.com> (visitado 10-10-2024).

- [16] Supabase, Inc. *Supabase – The Open Source Firebase Alternative*. Supabase, Inc. 2024. URL: <https://supabase.com> (visitado 10-10-2024).
- [17] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. The PostgreSQL Global Development Group. 2024. URL: <https://www.postgresql.org/> (visitado 14-04-2026).
- [18] MongoDB, Inc. *MongoDB: The Developer Data Platform*. MongoDB, Inc. 2024. URL: <https://www.mongodb.com/> (visitado 14-04-2026).
- [19] Software Freedom Conservancy. *Git*. Software Freedom Conservancy. 2024. URL: <https://git-scm.com/> (visitado 14-04-2026).
- [20] GitHub, Inc. *GitHub: Let's build from here*. GitHub, Inc. 2024. URL: <https://github.com/> (visitado 14-04-2026).
- [21] Figma, Inc. *Figma: The collaborative interface design tool*. Figma, Inc. 2024. URL: <https://www.figma.com/> (visitado 14-04-2026).
- [22] OpenJS Foundation. *ESLint: Find and fix problems in your JavaScript code*. OpenJS Foundation. 2024. URL: <https://eslint.org/> (visitado 14-04-2026).
- [23] Vercel, Inc. *Vercel – Build and deploy the best web experiences with the frontend cloud*. Vercel, Inc. 2024. URL: <https://vercel.com> (visitado 10-10-2024).
- [24] One.com. *One.com: Alojamiento web y dominios*. One.com. 2024. URL: <https://www.one.com/> (visitado 14-04-2026).
- [25] Ionic, Inc. *Capacitor – Cross-platform native runtime for web apps*. Ionic, Inc. 2024. URL: <https://capacitorjs.com> (visitado 10-10-2024).
- [26] Google LLC. *Firebase*. Google LLC. 2024. URL: <https://firebase.google.com/> (visitado 14-04-2026).
- [27] Amazon Web Services, Inc. *AWS Amplify*. Amazon Web Services, Inc. 2024. URL: <https://aws.amazon.com/amplify/> (visitado 14-04-2026).
- [28] Ken Schwaber y Jeff Sutherland. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. 2020. URL: <https://scrumguides.org/scrum-guide.html> (visitado 14-04-2026).
- [29] Jane Smith. «Título del Artículo». En: *Revista Ejemplo* 15.3 (2021), págs. 123-145. DOI: [10.1234/rev-ejemplo.2021.015](https://doi.org/10.1234/rev-ejemplo.2021.015).